

School Management App

Timetable Modul – Modal CRUD és Napi Csoportosítás

React 18 + Laravel 12 API | 2026

1. A Timetable modul egyedi jellemzői

A Timetable modul két új mintát vezet be amit az eddigi modulok nem használtak: a napok szerinti csoportosítást (reduce()) és a modal-alapú CRUD-ot (külön oldal helyett felugró ablak).

Jellemző	Korábbi modulok	Timetable modul
Létrehozás/szerkesztés helye	Külön oldal (/students/create)	Modal (felugró ablak) ugyanazon az oldalon
Lista megjelenítés	Táblázat, lapozással	3 oszlopos kártya elrendezés (nap szerint)
Adatcsoportosítás	Nem szükséges	reduce() nap szerint csoportosít

2. Napok szerinti csoportosítás – reduce()

A backend egy LAPOS listát ad vissza az összes órarendi bejegyzésről (rendezve nap és időpont szerint). A frontend feladata ezt átalakítani egy nap->bejegyzések struktúrára a kártyás megjelenítéshez.

```
const groupedByDay = (timetable ?? []).reduce((acc, entry) => {
  if (!acc[entry.day]) acc[entry.day] = [];
  acc[entry.day].push(entry);
  return acc;
}, {});

// Bemenet (lapos lista):
// [{day: 'monday', subject: 'Matek'}, {day: 'monday', subject: 'Fizika'},
//  {day: 'tuesday', subject: 'Töri'}]
//
// Kimenet (csoportosítva):
// {
//   monday: [{subject: 'Matek'}, {subject: 'Fizika'}],
//   tuesday: [{subject: 'Töri'}]
// }
```

Miért reduce() és nem 6x filter()?

Egy naiv megközelítés minden naphoz külön filter()-t hívna: timetable.filter(e => e.day === 'monday'), timetable.filter(e => e.day === 'tuesday'), stb. Ez 6 KÜLÖN végigfutást jelentene UGYANAZON a listán. A reduce() EGYETLEN végigfutással (O(n) komplexitás) csoportosítja az összes elemet egyszerre - jelentősen hatékonyabb, különösen nagyobb órarendeknél.

Az acc[entry.day] = [] minta - defenzív inicializálás

A if (!acc[entry.day]) acc[entry.day] = []; sor ELLENŐRZI hogy létezik-e már tömb az adott naphoz - ha ez az ELSŐ bejegyzés arra a napra, létrehoz egy üres tömböt, mielőtt a push()-t hívnánk. Enélkül a acc[entry.day].push(entry) hibát dobna undefined-en végzett push() miatt, ha még nem volt korábbi bejegyzés ugyanarra a napra.

3. Modal minta – felugró ablak külön oldal helyett

A TimetableEntryModal komponens egy 'egy komponens, két mód' mintát követ, hasonlóan a StudentFormPage-hez - de OLDAL helyett MODAL formájában jelenik meg, mert az órarendi bejegyzés form-ja kicsi és gyors kitöltésű, nem indokol teljes oldalváltást.

3.1 A modalAction state - két információ egy objektumban

```
const [modalAction, setModalAction] = useState(null);

// Új óra hozzáadása hétfőre:
setModalAction({ day: 'monday', entry: null });

// Meglévő óra szerkesztése:
setModalAction({ day: entry.day, entry: entry });

// Modal bezárása:
setModalAction(null);
```

Miért egy objektum és nem két külön state (showModal, editingEntry)?

Két külön state (pl. const [showModal, setShowModal] = useState(false) és const [editingEntry, setEditingEntry] = useState(null)) esetén EGYSZERRE kellene mindkettőt szinkronban tartani minden művelethez - könnyen előfordulhatna hogy showModal=true de editingEntry még a régi értéket tartalmazza. Egyetlen modalAction objektummal (null = zárva, {day, entry} = nyitva) a modal állapota MINDIG konzisztens egyetlen állítással.

3.2 A modal renderelése feltételesen

```

{modalAction && (
  <TimetableEntryModal
    classId={classId}
    day={modalAction.day}
    entry={modalAction.entry}
    onClose={closeModal}
    onSave={handleSave}
    onDelete={handleDelete}
  />
)}

```

A && feltételes renderelés mintája

A `{modalAction && (<Component />)}` egy gyakori React minta: ha `modalAction` null/false/undefined, a `&&` kifejezés RÖVIDRE ZÁRUL és semmi nem jelenik meg. Ha `modalAction` egy objektum (igaz értékű), a JSX renderelődik. Ez tisztább mint egy teljes `if-else` blokk egy egyszerű megjelenítsd-vagy-ne döntéshez.

4. Egy komponens, két mód – a modalon belül

```

export default function TimetableEntryModal({ classId, day, entry, ... })
{
  const isEditMode = Boolean(entry);

  useEffect(() => {
    if (isEditMode) {
      reset({
        subject_id: entry.subject?.id ?? '',
        teacher_id: entry.teacher?.id ?? '',
        start_time: entry.start_time ?? '',
        // ...
      });
    } else {
      reset({ subject_id: '', teacher_id: '', start_time: '', ...
    });
  }
  }, [isEditMode, entry, reset]);

```

Miért kell az ELSE ág is a reset()-nél?

A `StudentFormPage`-nél nem volt szükség `else` ágra, mert ott a form MINDIG üresen indul `create` módban (a komponens újra mountolódik route váltáskor). Itt viszont a MODAL ugyanazon az oldalon nyílik újra és újra különböző módokban (előbb szerkesztés egy hétfői órát, bezárod, majd új órát adsz hozzá keddre) - a komponens NEM mountolódik újra, csak a props változik. Az `else` ág biztosítja hogy váltáskor a form ÜRESRE álljon vissza, ne maradjon benne az előző szerkesztett óra adata.

5. Mentés – day és class_id hozzáfűzése a payloadhoz

```
function onSubmit(data) {
  onSave({
    ...data,
    class_id: Number(classId),
    day,
    teacher_id: data.teacher_id || null,
  });
}
```

Miért nincs class_id és day mező magában a form-ban?

A class_id-t és a day-t a SZÜLŐ komponens (TimetablePage) már ismeri - a felhasználó ELŐZŐLEG kiválasztotta az osztályt egy dropdown-ban, és a nap magából abból derül ki, melyik oszlop '+' gombjára kattintott. Felesleges lenne ezeket ÚJRA elkérni a modal form-jában - a props-ként érkező classId és day értékeket egyszerűen HOZZÁFÜZZÜK a form saját mezőihöz (subject_id, teacher_id, start_time stb.) mentés előtt.

6. Update hook egy apró trükkje

```
const updateTimetable = useUpdateTimetable(modalAction?.entry?.id);
```

Miért működik ez akkor is, ha modalAction null?

A modalAction?.entry?.id egy LÁNCOLT optional chaining - ha modalAction null, az egész kifejezés undefined lesz (nem dob hibát). A useUpdateTimetable(undefined) hook ilyenkor 'üresen' létrejön, de nem hívjuk meg ténylegesen amíg a modal nincs nyitva és a felhasználó nem submitol - a React Hooks szabályai szerint a hook mindig meg kell hívni minden renderben (feltétel nélkül), csak a PARAMÉTERE lehet átmenetileg érvénytelen.

7. Összefoglaló – mikor válasszunk modalt oldal helyett?

Modal (mint Timetable)	Külön oldal (mint Students)
Kevés mező (4-6 alatt)	Sok mező, hosszabb form
A kontextus (osztály, nap) már adott	Önálló entitás, nincs szükség külső kontextusra
Gyors, ismétlődő művelet (sok órát rögzítünk egymás után)	Ritkább, egyszeri művelet

A felhasználó a listaoldalon akar maradni

Természetes navigációs váltás elfogadható

– *School Management App fejlesztési dokumentáció* –